MANUAL OPERATIVO

OpenVPN en AWS con Terraform

GitHub · macOS · Instalación · Configuración · Usuarios

Alcance: Este manual parte de la plantilla existente con main.tf, variables.tf, outputs.tf, terraform.tfvars.example y README.md. Incluye el flujo para versionarla en GitHub desde un Mac y reutilizarla para futuros clientes.

Versión	1.0
Plataforma	AWS + Terraform + OpenVPN
Equipo local	macOS
Fecha	09-sep-2026

1. Objetivo y arquitectura de trabajo

El objetivo es mantener una plantilla reutilizable para desplegar un servidor OpenVPN en una cuenta AWS mediante Terraform. La plantilla crea la infraestructura necesaria, instala y configura OpenVPN en una instancia Ubuntu y permite generar perfiles .ovpn para los usuarios.

Importante: El repositorio debe contener únicamente código y documentación. Nunca se deben subir credenciales AWS, llaves .pem, archivos .ovpn ni el archivo terraform.tfvars con valores de clientes.

2. Estructura recomendada del repositorio

La estructura actual es válida para iniciar. Para uso recurrente con clientes, se recomienda mantenerla así:

```
openvpn-terraform-ubuntu26/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars.example
├── README.md
├── .gitignore
└── docs/
    └── Manual_OpenVPN_Terraform.pdf
```

Archivo	Uso
main.tf	Recursos AWS y lógica principal del despliegue.
variables.tf	Declaración de variables de entrada.
outputs.tf	Comandos, IPs y salidas útiles del despliegue.
terraform.tfvars.example	Ejemplo de parámetros sin datos sensibles.
README.md	Guía rápida de uso del repositorio.
.gitignore	Evita subir estado Terraform, llaves, perfiles VPN y variables reales.
docs/	Documentación operativa y manuales.

3. Subir el proyecto a GitHub desde macOS

3.1 Abrir Terminal y entrar a la carpeta

```
cd /ruta/donde/esta/openvpn-terraform-ubuntu26
ls -la
```

Debes ver main.tf, variables.tf, outputs.tf, terraform.tfvars.example y README.md.

3.2 Crear .gitignore antes del primer commit

```
cat > .gitignore <<'EOF'
# Terraform local state
.terraform/
*.tfstate
*.tfstate.*
.terraform.lock.hcl
crash.log
crash.*.log

# Variables reales / secretos
terraform.tfvars
*.auto.tfvars
*.auto.tfvars.json
.env
.env.*

# Credenciales y perfiles VPN
*.pem
*.key
*.crt
*.ovpn
client-configs/

# macOS / editores
.DS_Store
.vscode/
.idea/
EOF
```

Nota: En equipos de trabajo suele ser conveniente versionar .terraform.lock.hcl para fijar versiones de providers. Si decides hacerlo, elimina esa línea del .gitignore.

3.3 Validar que no existan secretos

```
grep -RniE 'AKIA|aws_secret_access_key|BEGIN.*PRIVATE KEY|password|secret' . --exclude-
dir=.git || true
```

Revisa manualmente cualquier coincidencia antes de publicar el repositorio.

3.4 Inicializar Git y crear el primer commit

```
git init
git add .
git status
git commit -m "feat: plantilla inicial OpenVPN con Terraform"
```

3.5 Crear el repositorio en GitHub

En GitHub crea un repositorio nuevo, por ejemplo openvpn-terraform-aws. Si el código se reutilizará para clientes, se recomienda mantenerlo privado y no inicializarlo con README, .gitignore o licencia para evitar conflictos con los archivos locales.

3.6 Vincular el repositorio local y subirlo

```
git branch -M main
git remote add origin https://github.com/ORGANIZACION/openvpn-terraform-aws.git
git push -u origin main
```

Autenticación GitHub: GitHub no acepta la contraseña normal de la cuenta para Git por HTTPS. En macOS puedes autenticarte con GitHub CLI (gh auth login), un Personal Access Token o una llave SSH.

3.7 Flujo recomendado para cambios futuros

```
git checkout -b feature/mejora-openvpn
# realizar cambios
git add .
git commit -m "feat: describe el cambio"
git push -u origin feature/mejora-openvpn
```

Después crea un Pull Request hacia main. Esto ayuda a revisar cambios antes de afectar la plantilla base.

4. Reutilizar la plantilla para un nuevo cliente

Para un nuevo despliegue, clona el repositorio en lugar de copiar carpetas manualmente:

```
git clone https://github.com/ORGANIZACION/openvpn-terraform-aws.git
cd openvpn-terraform-aws
```

Crea el archivo de variables del cliente a partir del ejemplo:

```
cp terraform.tfvars.example terraform.tfvars
```

5. Prerrequisitos en el Mac

- Terraform instalado.
- AWS CLI instalado.
- Git instalado.
- Acceso programático o SSO/perfil AWS válido.
- Permisos para crear EC2, Security Groups, Elastic IP y Key Pair.
- VPC existente y subnet pública existente.
- Acceso a Internet desde el equipo local.

```
terraform -version
aws --version
git --version
```

6. Configurar acceso a AWS

No coloques Access Key ni Secret Key dentro de main.tf. Configura las credenciales mediante AWS CLI o el mecanismo corporativo definido para la cuenta.

```
aws configure --profile openvpn
export AWS_PROFILE=openvpn
aws sts get-caller-identity
```

7. Configurar terraform.tfvars

Edita terraform.tfvars con los datos de la cuenta y la red del cliente. Basado en la plantilla actual, los parámetros principales son:

```
aws_region = "us-east-1"
vpc_id = "vpc-xxxxxxxxxxxxxxxxx"
vpc_cidr = "10.60.0.0/16"
subnet_id = "subnet-xxxxxxxxxxxxxxxxx"
name_prefix = "BastionOpenVPN"
key_name = "BastionOpenVPN"
instance_type = "t3.micro"
openvpn_allowed_cidrs = ["203.0.113.10/32"]
initial_client_name = "cliente1"
create_eip = true
```

Seguridad: Evita usar 0.0.0.0/0 en openvpn_allowed_cidrs cuando conozcas las IP públicas autorizadas. Para SSH, permite únicamente las IPs administrativas necesarias.

8. Despliegue con Terraform

Inicializar

```
terraform init
```

Formatear

```
terraform fmt -recursive
```

Validar sintaxis

```
terraform validate
```

Revisar plan

```
terraform plan
```

Aplicar

```
terraform apply
```

En terraform apply confirma con yes únicamente después de validar región, VPC, subnet, Security Group, IPs permitidas y tipo de instancia.

9. Acceso al servidor y descarga del primer perfil

La plantilla actual genera una llave SSH local y expone outputs para facilitar la conexión.

```
chmod 400 BastionOpenVPN.pem
terraform output ssh_command
```

Ejemplo esperado:

```
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
```

Para descargar el perfil inicial:

```
terraform output download_client_ovpn_command
```

Ejemplo:

```
scp -i BastionOpenVPN.pem ubuntu@IP_PUBLICA:~/client-configs/files/cliente1.ovpn .
```

10. Validaciones del servidor OpenVPN

```
sudo systemctl status openvpn-server@server
sudo ss -lunp | grep 1194
sudo cat /etc/openvpn/server/server.conf
ls -lh ~/client-configs/files/
```

El servicio debe aparecer activo y el puerto UDP 1194 debe estar en escucha.

11. Creación de usuarios OpenVPN

La plantilla dispone del script crear-cliente-openvpn.sh. Cada usuario debe tener su propio perfil .ovpn; no reutilices el mismo archivo entre personas.

11.1 Crear un usuario

```
~/crear-cliente-openvpn.sh nombre_usuario
```

Ejemplo:

```
~/crear-cliente-openvpn.sh carlos.rojas
```

El perfil se genera en:

```
/home/ubuntu/client-configs/files/carlos.rojas.ovpn
```

11.2 Descargar el perfil al Mac

```
scp -i BastionOpenVPN.pem ubuntu@IP_PUBLICA:~/client-configs/files/carlos.rojas.ovpn .
```

Importa el archivo en OpenVPN Connect y realiza una prueba de conexión.

11.3 Registro operativo de usuarios

Usuario	Fecha	Responsable	Perfil entregado	Estado

12. Revocación o eliminación de usuarios

El manual original solo documenta la creación de perfiles; no define un procedimiento de revocación. Por seguridad, la operación de baja debe implementarse con la PKI/Easy-RSA utilizada por el servidor, regenerando y publicando la CRL cuando corresponda. Antes de estandarizar este paso, valida cómo quedó construido crear-cliente-openvpn.sh y la configuración de certificados de tu main.tf.

Recomendación de plantilla: Añade en una siguiente versión un script revocar-cliente-openvpn.sh y documenta la actualización de la CRL. Así el alta y la baja de usuarios quedan automatizadas y repetibles.

13. Buenas prácticas de seguridad

- No subir terraform.tfvars al repositorio.
- No subir *.pem, *.ovpn, claves privadas, certificados privados ni archivos de estado Terraform.
- Mantener el repositorio privado si contiene arquitectura o configuración reutilizable de clientes.
- Restringir SSH a IPs administrativas específicas.
- Restringir UDP 1194 a rangos autorizados cuando sea viable.
- Crear un perfil .ovpn diferente por usuario.
- Documentar altas, bajas y responsable de cada perfil.
- Rotar o revocar perfiles cuando una persona deja de requerir acceso.
- Usar ramas y Pull Requests para cambios sobre la plantilla base.
- Validar terraform plan antes de cada apply.

14. Flujo operativo recomendado por cliente

1. Clonar el repositorio base.

2. Crear terraform.tfvars a partir del ejemplo.

3. Configurar AWS_PROFILE.

4. Ejecutar terraform init / validate / plan.

5. Revisar reglas de red y Security Groups.

6. Ejecutar terraform apply.

7. Validar servicio OpenVPN.

8. Crear perfiles individuales de usuarios.

9. Entregar perfiles por canal seguro.

10. Registrar usuarios y accesos.

11. Guardar cambios genéricos de código mediante branch/PR; nunca subir datos específicos del cliente.

15. Destrucción del ambiente

```
terraform destroy
```

Revisa el plan de destrucción y confirma con yes. Antes de eliminar, asegúrate de que el cliente ya no depende de la VPN y conserva cualquier evidencia operativa necesaria.

16. Comandos rápidos de diagnóstico

Validación	Comando
AWS identity	aws sts get-caller-identity
Terraform state	terraform state list
Outputs	terraform output
OpenVPN service	sudo systemctl status openvpn-server@server
OpenVPN logs	sudo journalctl -u openvpn-server@server --no-pager -n 100
UDP 1194	sudo ss -lunp | grep 1194
Perfiles	ls -lh ~/client-configs/files/

17. Checklist antes de reutilizar la plantilla

[ ] El repositorio no contiene secretos.

[ ] .gitignore protege tfvars, estados, llaves y perfiles VPN.

[ ] terraform.tfvars.example usa valores ficticios.

[ ] README explica prerequisitos y despliegue.

[ ] El Security Group no expone SSH innecesariamente.

[ ] Cada usuario tiene un .ovpn individual.

[ ] Existe registro de usuarios activos.

[ ] Se ha definido un procedimiento de revocación antes de producción.